Co-evoluzione tra opinioni, topologia e fact-checking in una rete sociale simulata con agenti LLM

Relazione di progetto – Analisi di Social Network e Media

Stefano Brusco

Corso di Laurea Magistrale in Ingegneria Informatica

Università della Calabria

13 ottobre 2026

Sommario

Il progetto realizza una simulazione in cui le opinioni di agenti basati su un modello linguistico (LLM), la struttura della rete che li collega e un intervento di fact-checking evolvono insieme, in un unico ciclo. La simulazione è eseguita su un sottografo di 5 000 nodi della rete di co-autoraggio `ogbl-collab`. La diffusione delle opinioni segue un modello Linear Threshold esteso, una rete neurale su grafi (GraphSAGE) guida la modifica dei legami e una procedura greedy sceglie i nodi da convertire in fact-checker. La relazione descrive le scelte progettuali, motivandole rispetto ad alcune alternative, e analizza i risultati: la diffusione si arresta a circa il 40% della rete, la segregazione aumenta soprattutto sul piano delle opinioni, e un intervento con 20 fact-checker protegge una parte dei nodi non ancora contagiati, senza ridurre il numero di infetti.

Indice

1 Introduzione **3**

2 Dati e preparazione della rete **3**
2.1 Il dataset . . . 3
2.2 Campionamento del sottografo . . . 3
2.3 Rilevamento delle comunità . . . 4

3 Architettura del sistema **4**
3.1 Tre livelli in un ciclo chiuso . . . 4
3.2 Svolgimento di uno step . . . 5
3.3 Aspetti implementativi . . . 5

4 Scelte algoritmiche **6**
4.1 Modello di diffusione . . . 6
4.2 Link prediction con GraphSAGE . . . 6
4.3 Rewiring a densità costante . . . 7
4.4 Selezione dei fact-checker . . . 7
4.5 Metriche . . . 7

5 Esperimenti e risultati **8**
5.1 Configurazione dell'esperimento . . . 8
5.2 Diffusione . . . 8
5.3 Segregazione . . . 9
5.4 Effetto dell'intervento . . . 10
5.5 Tempi di esecuzione . . . 10

6 Limiti e sviluppi futuri **11**

7 Conclusioni **11**

1 Introduzione

Nelle reti sociali online gli utenti tendono a raggrupparsi con chi la pensa allo stesso modo, formando le cosiddette *echo chamber* [4]. Come visto nel corso, la correlazione tra legami e opinioni può avere due origini: la *selezione*, per cui si scelgono contatti simili, e l'*influenza sociale*, per cui si modificano le proprie opinioni per avvicinarsi a quelle dei contatti [18, 1]. Nella realtà i due meccanismi agiscono insieme: la rete condiziona le opinioni e le opinioni modificano la rete. In questo caso si parla di co-evoluzione [9].

I modelli classici trattano di solito questi aspetti separatamente. I modelli di diffusione come Independent Cascade e Linear Threshold [11] considerano un grafo fisso, mentre i modelli di co-evoluzione rappresentano l'opinione con una variabile semplice. In entrambi i casi la reazione di un individuo a ciò che legge viene ridotta a un parametro numerico. Il progetto prova a sostituire questo parametro con un agente basato su un LLM, che legge i messaggi pubblicati dai vicini e decide come reagire, sul modello degli agenti generativi proposti in [21].

Gli obiettivi del progetto sono tre:

- realizzare un sistema in cui agenti, rete e intervento siano collegati in un ciclo chiuso, invece di essere eseguiti come fasi indipendenti;
- misurare come cambia la segregazione della rete, distinguendo la struttura delle comunità dalla distribuzione delle opinioni;
- stimare l'effetto di un intervento di fact-checking con un numero limitato di nodi, confrontandolo con una simulazione senza intervento.

La relazione è organizzata come segue. La Sezione 2 descrive il dataset e la preparazione della rete; la Sezione 3 presenta l'architettura del sistema; la Sezione 4 motiva le principali scelte algoritmiche; la Sezione 5 riporta i risultati; le Sezioni 6 e 7 discutono i limiti e le conclusioni.

2 Dati e preparazione della rete

2.1 Il dataset

Il dataset utilizzato è `ogbl-collab` dell'Open Graph Benchmark [10], una rete di co-autoraggio accademico con 235 868 autori. Ogni nodo ha un vettore di 128 feature ottenuto con Word2Vec dai testi delle pubblicazioni, che descrive i temi di ricerca dell'autore. Questi vettori sono stati usati come rappresentazione iniziale dei nodi.

Una rete di co-autoraggio non è una rete sociale online, ma ne condivide alcune caratteristiche importanti per il progetto: comunità ben definite, un coefficiente di clustering elevato e una distribuzione dei gradi a coda pesante.

2.2 Campionamento del sottografo

Simulare un agente LLM per ciascuno dei circa 235 000 nodi non era possibile con le risorse disponibili, quindi è stato estratto un sottografo di 5 000 nodi. Il codice permette di scegliere tra quattro strategie di campionamento: selezione casuale di nodi, visita in ampiezza (BFS) a partire da un hub, random walk e Forest Fire. La selezione casuale produce un grafo frammentato e molto meno denso; la BFS tende a includere solo il nucleo denso intorno al nodo di partenza; il random walk privilegia i nodi ad alto grado. È stato quindi scelto il Forest Fire, che secondo Leskovec e Faloutsos [15] è tra i metodi che conservano meglio le proprietà della rete originale. Il parametro di propagazione è stato fissato a 0.5.

La Tabella 1 confronta il sottografo con la componente connessa principale della rete completa.

Tabella 1: Confronto tra la rete originale (componente connessa principale) e il sottografo campionato.

Metrica	**Rete originale**	**Sottografo**
Nodi	232 865	5 000
Archi	961 883	26 246
Grado medio	8.3	10.5
Grado massimo	382	119
Clustering medio	0.720	0.699
Modularità (Label Propagation)	0.696	0.827

Il clustering è conservato quasi esattamente, mentre il grado massimo si riduce molto: il campione perde gli hub più estremi della rete originale, e di questo bisogna tenere conto nell'interpretare i risultati.

2.3 Rilevamento delle comunità

Per individuare le comunità è stata usata la Label Propagation [22], preferita a Louvain [2] perché ha un costo quasi lineare e non richiede parametri. Il suo difetto principale è che due esecuzioni possono dare risultati diversi; per questo la partizione è stata calcolata una sola volta e salvata. Sul sottografo sono state trovate 415 comunità, con modularità $Q = 0.827$. Questa partizione resta fissa per tutta la simulazione e serve come riferimento per misurare la segregazione strutturale.

3 Architettura del sistema

3.1 Tre livelli in un ciclo chiuso

Il sistema è diviso in tre livelli. Il *livello cognitivo* contiene gli agenti LLM e la macchina a stati che decide le loro transizioni. Il *livello strutturale* contiene la rete neurale su grafi (GraphSAGE), che stima la probabilità dei legami, e il meccanismo di rewiring che modifica la rete. Il *livello di intervento* sceglie i nodi da convertire in fact-checker e li inserisce nella simulazione. I tre livelli non sono eseguiti uno dopo l'altro, ma a ogni passo di simulazione (step): le opinioni prodotte allo step t modificano la rete dello step $t + 1$, che a sua volta determina che cosa leggerà ciascun agente. Se la diffusione fosse simulata prima su una rete fissa e la rete modificata solo dopo, questa interazione andrebbe persa. Lo schema complessivo è riportato in Figura 1.

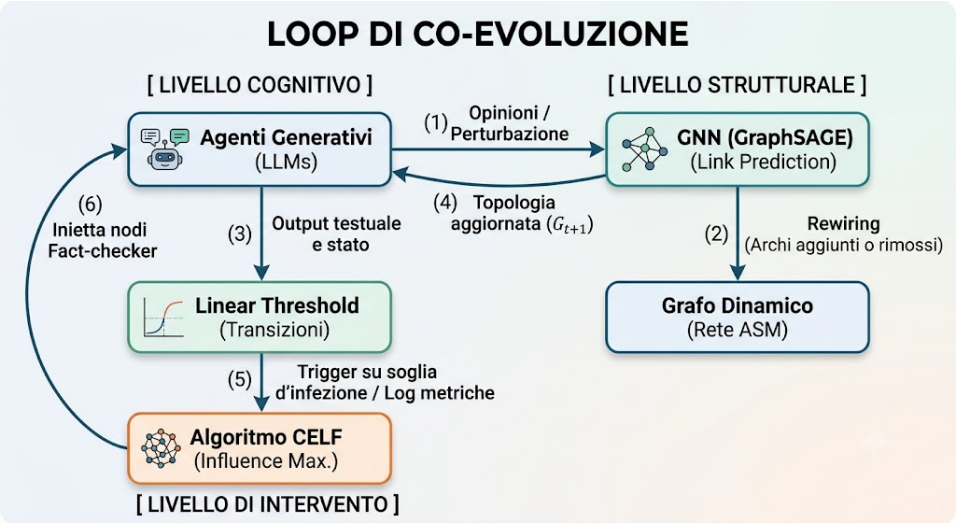


Figura 1: Schema del ciclo di co-evoluzione tra i tre livelli. Il blocco indicato come CELF corrisponde al modulo `celf.py`; nell'esperimento è stata usata la selezione descritta nella Sezione 4.4.

3.2 Svolgimento di uno step

Ogni step della simulazione si svolge in sei fasi:

1. vengono estratti casualmente il 15% dei nodi (750 agenti), che sono gli unici attivi in quello step;
2. ogni agente attivo riceve un prompt con il proprio profilo, il proprio stato e gli ultimi messaggi dei vicini, e l'LLM risponde in formato JSON con un'opinione testuale, un livello di suscettibilità da 0 a 1 e uno stato proposto;
3. la macchina a stati decide la transizione, combinando la risposta dell'LLM con la frazione di vicini infetti (Sezione 4.1);
4. l'embedding del nodo viene spostato in una direzione che dipende dal nuovo stato;
5. GraphSAGE viene aggiornato per 15 epoche sulla rete corrente;
6. ogni due step, il rewiring sostituisce fino a 50 archi.

Al termine di ogni step vengono calcolate le metriche e salvato un checkpoint.

3.3 Aspetti implementativi

La parte più costosa della simulazione sono le chiamate all'LLM. Per contenere i tempi sono state fatte alcune scelte:

- il modello è Llama 3 8B Instruct [6], quantizzato a 4 bit, servito in locale con vLLM [14], che elabora in parallelo fino a 150 richieste. Rispetto a un'API cloud non ci sono limiti di frequenza né costi per token, e la versione del modello resta sotto controllo;
- solo il 15% dei nodi è attivo a ogni step; questo riduce le chiamate e rappresenta il fatto che non tutti gli utenti sono attivi nello stesso momento;
- tutti gli agenti attivi leggono lo stesso stato della rete (aggiornamento sincrono), così il risultato non dipende dall'ordine in cui vengono elaborati e le richieste possono partire in parallelo;
- ogni richiesta all'LLM usa un seed ricavato dal testo del prompt: due simulazioni che inviano lo stesso prompt ricevono la stessa risposta, cosa indispensabile per il confronto tra intervento e controllo;

- poiché le sessioni Kaggle durano al massimo 12 ore, a ogni step viene salvato un checkpoint completo, che permette di riprendere la simulazione nella sessione successiva.

4 Scelte algoritmiche

4.1 Modello di diffusione

Ogni agente può trovarsi in quattro stati: suscettibile (S), infetto (I), cioè convinto della narrazione polarizzante, resistente (R) e fact-checker (F). La struttura riprende i modelli epidemici visti nel corso [12, 1], con l'aggiunta dello stato F.

Le transizioni seguono un modello Linear Threshold [11]: un nodo si infetta quando la frazione dei suoi vicini infetti supera una soglia. Rispetto al modello classico sono state introdotte due modifiche. La prima è che la soglia di ogni nodo non è fissa, ma viene modulata dalla suscettibilità s dichiarata dall'agente:

$$\theta_v^{\text{eff}} = \theta_v \, (1.5 - s), \qquad \theta_v \sim \mathcal{N}(0.3,\ 0.1^2). \tag{1}$$

Con $s = 1$ la soglia si dimezza, con $s = 0$ aumenta del 50%. La seconda è che il contagio avviene solo se la soglia è superata *e* l'agente propone di diventare infetto: la struttura della rete resta quindi una condizione necessaria, e l'LLM non può infettare un nodo che non ha abbastanza vicini infetti. Le regole sono riassunte nella Tabella 2, dove f^I e f^F indicano la frazione di vicini infetti e fact-checker.

Tabella 2: Regole di transizione degli agenti.

Transizione	**Condizione**
$S \to R$	$f^F \geq 0.10$, $f^F \geq f^I$ e l'agente non propone I (protezione da fact-checking)
$S \to I$	$f^I \geq \theta^{\text{eff}}$ e l'agente propone I
$S \to R$	almeno il 12% di vicini infetti, suscettibilità bassa e nessuna intenzione di diffondere
$I \to R$	$f^F \geq 0.25$, oppure l'agente cambia idea da solo
$R \to I$	ricaduta, rara: $f^I \geq 0.6$ e suscettibilità maggiore di 0.8

Sono state considerate due alternative. Il modello Independent Cascade descrive un contagio «semplice», in cui basta un singolo contatto per essere attivati; per l'adozione di un'opinione sembra più realistico un contagio «complesso», che richiede il rinforzo di più contatti [3], ed è esattamente ciò che rappresentano i modelli a soglia [5]. I modelli SIR, invece, descrivono l'andamento medio delle popolazioni senza usare la rete, che in questo progetto è proprio l'oggetto di studio.

4.2 Link prediction con GraphSAGE

Per collegare le opinioni alla struttura della rete serviva un canale. Quello scelto è il vettore di embedding di ogni nodo: quando un agente cambia stato, il suo embedding viene spostato lungo una direzione comune a tutti gli agenti (in un verso per gli infetti, nel verso opposto per resistenti e fact-checker), con una piccola componente individuale. In questo modo nodi con la stessa opinione diventano più simili nello spazio degli embedding.

Su questi embedding lavora una rete GraphSAGE [8] a due layer con aggregatore di media, che calcola per ogni coppia di nodi un punteggio di probabilità del legame. Il modello è addestrato con l'entropia incrociata binaria su archi esistenti e coppie casuali, e a ogni step viene aggiornato per 15 epoche partendo dai parametri dello step precedente. Durante la simulazione la loss scende da 0.816 a 0.360, sotto il valore $\ln 2 \approx 0.693$ di un classificatore che risponde a caso.

GraphSAGE è stato preferito alle alternative per tre motivi. Le euristiche classiche come il numero di vicini comuni [17] e gli embedding come node2vec [7] non usano le feature dei nodi, quindi non vedrebbero le opinioni. GraphSAGE è induttivo: i parametri appresi restano validi anche quando la rete cambia, e non serve riaddestrare il modello da zero a ogni step. Infine, a differenza della GCN [13], tratta separatamente il nodo e i suoi vicini, ed è utile perché l'opinione è contenuta proprio nell'embedding del nodo. GAT [23] avrebbe aggiunto un meccanismo di attenzione più costoso, non necessario per gli obiettivi del progetto.

4.3 Rewiring a densità costante

Il rewiring usa i punteggi di GraphSAGE per modificare la rete. Ogni due step vengono considerate coppie di nodi a distanza due, cioè nodi che hanno almeno un vicino in comune, come nella chiusura triadica [1]. Le coppie con il punteggio più alto diventano nuovi archi (al massimo 50), e per ogni arco aggiunto viene rimosso l'arco esistente con il punteggio più basso. In questo modo il numero di archi resta costante, come nel modello di Holme e Newman [9].

La scelta di mantenere costante la densità deriva da un problema riscontrato in una prima versione, che rimuoveva solo gli archi con punteggio sotto una soglia. Il link predictor, essendo addestrato sugli archi esistenti, assegna loro punteggi alti, quindi in pratica non veniva rimosso nulla e la rete diventava sempre più densa. Con lo scambio, invece, ogni variazione delle metriche dipende da come vengono redistribuiti i legami e non da un aumento della densità.

4.4 Selezione dei fact-checker

Il progetto prevedeva inizialmente di scegliere i fact-checker con l'algoritmo CELF [16], una versione ottimizzata del greedy per l'Influence Maximization. Durante lo sviluppo si è però visto che CELF non era adatto a questo caso.

L'effetto di un fact-checker dipende dalle soglie della Tabella 2: un suscettibile è protetto quando almeno il 10% dei suoi vicini è fact-checker, un infetto viene recuperato quando lo è almeno il 25%. La funzione da massimizzare è quindi il numero di nodi che superano queste soglie. Si consideri però un nodo infetto con 8 vicini: per recuperarlo servono 2 fact-checker. Aggiungere il primo non produce alcun guadagno, aggiungere il secondo produce un guadagno di 1. Il guadagno aumenta invece di diminuire, quindi la funzione non è submodulare. CELF si basa proprio sulla submodularità per evitare di ricalcolare tutti i guadagni [1], e in questo caso potrebbe scartare il candidato migliore. Applicare CELF al modello Independent Cascade avrebbe dato la garanzia teorica di approssimazione $1 - 1/e$ [11], ma su una funzione diversa da quella che governa la simulazione.

Si è quindi usato un greedy semplice, che a ogni iterazione ricalcola il guadagno esatto di tutti i candidati. Poiché il guadagno di un nodo dipende solo dai suoi vicini, il calcolo è veloce: circa un milione di operazioni per scegliere 20 nodi. I candidati sono solo i nodi suscettibili, perché convertire un infetto in fact-checker non è realistico e convertire un resistente sarebbe inutile. Il limite di questo approccio è che considera solo l'effetto immediato dei fact-checker, senza tenere conto di come la rete evolverà dopo; per questo il risultato è stato controllato a posteriori (Sezione 5.4).

4.5 Metriche

Per misurare la segregazione sono state usate due famiglie di metriche.

- *Metriche strutturali*: la modularità Q [20] e l'Echo Chamber Index (ECI), cioè la frazione media di vicini di un nodo che appartengono alla sua stessa comunità. Entrambe sono calcolate rispetto alla partizione iniziale.
- *Metriche di opinione*, con la codifica S= 0, I= +1, R=F= −1: l'*opinion homophily* H, cioè la frazione di archi tra nodi con la stessa opinione; la *belief assortativity* r, cioè la

correlazione tra le opinioni ai due estremi degli archi [19]; la *belief polarisation*, cioè la varianza delle opinioni normalizzata.

Le due famiglie servono perché misurano cose diverse: le metriche strutturali non dipendono dalle opinioni e non si accorgerebbero di gruppi di persone con la stessa opinione che appartengono a comunità diverse.

5 Esperimenti e risultati

5.1 Configurazione dell'esperimento

La simulazione è stata eseguita su Kaggle con due GPU T4. Il 15% dei nodi (750, scelti tra i più centrali con una combinazione di PageRank, centralità di Katz e grado) parte nello stato infetto. La Fase 2, cioè la co-evoluzione vera e propria, dura 100 step (0–99) ed è stata divisa in due sessioni da 50 step. Allo step 99 è stato salvato un checkpoint, da cui sono partite due simulazioni di 30 step (100–129): una con 20 fact-checker e una di controllo senza intervento. Un semplice confronto «prima e dopo» avrebbe mescolato l'effetto dell'intervento con l'evoluzione spontanea della rete; confrontando le due simulazioni allo stesso step, invece, la differenza è attribuibile all'intervento. All'inizio della Fase 3 gli infetti erano il 40.6% della rete, sopra la soglia di attivazione dell'intervento fissata a 0.4.

5.2 Diffusione

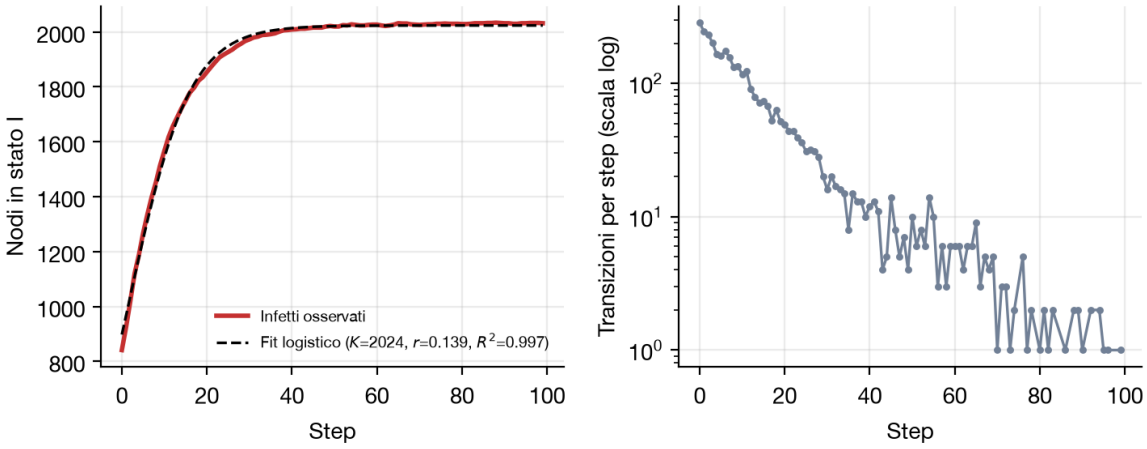


Figura 2: A sinistra, il numero di infetti durante la Fase 2 e la curva logistica stimata. A destra, il numero di transizioni per step in scala logaritmica.

Durante la Fase 2 gli infetti crescono da 841 a 2 032, i suscettibili scendono da 3 965 a 978 e i resistenti salgono da 194 a 1 990. Come si vede in Figura 2, l'andamento degli infetti è ben descritto da una curva logistica ($R^2 = 0.997$), che si stabilizza intorno a 2 024 nodi, cioè il 40.5% della rete. Il numero di transizioni per step cala rapidamente: il 78% avviene nei primi 20 step e dopo lo step 40 la situazione è quasi stabile.

Il risultato è simile a quello del modello SIR visto nel corso: nel modello SI l'infezione raggiunge tutta la popolazione, mentre qui si ferma molto prima, perché una parte dei nodi diventa resistente e perché alcuni suscettibili hanno una soglia che non viene mai raggiunta.

5.3 Segregazione

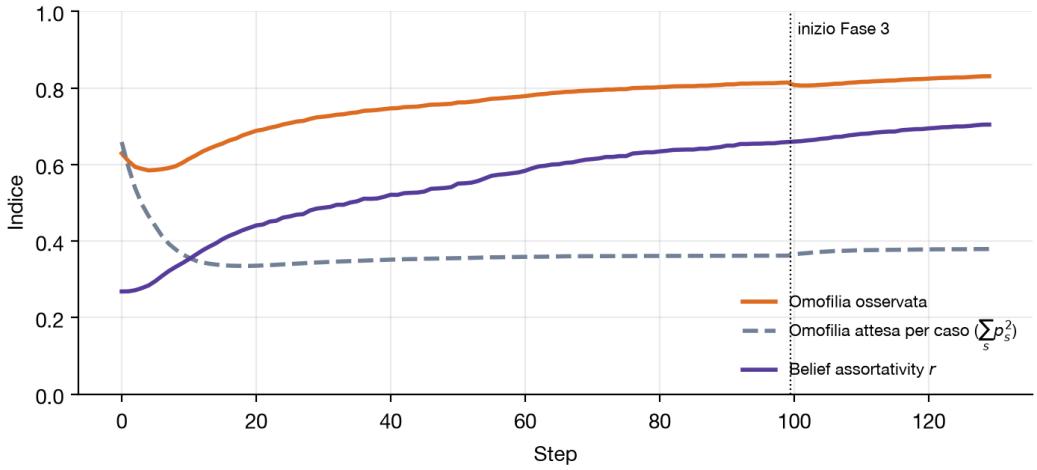


Figura 3: Omofilia di opinione osservata, omofilia attesa se gli stati fossero assegnati a caso e belief assortativity, dallo step 0 allo step 129.

Tra lo step 0 e lo step 99 le metriche strutturali crescono poco: ECI +5.0% e modularità +6.0%. Le metriche di opinione crescono molto di più: l'opinion homophily del 29.4% e la belief assortativity del 145.5%, da 0.268 a 0.659.

Per capire se l'omofilia è reale si è usato il test visto nel corso [1], che confronta la frazione di archi tra opinioni diverse con quella che ci si aspetterebbe se le opinioni fossero distribuite a caso. All'inizio il rapporto tra osservato e atteso è 1.087, quindi non c'è omofilia di opinione; allo step 99 scende a 0.291, cioè gli archi tra opinioni diverse sono meno di un terzo di quelli attesi. L'omofilia viene quindi prodotta dalla dinamica della simulazione.

La Figura 3 mostra anche perché è utile confrontare l'omofilia osservata con quella attesa. Nei primi step l'omofilia osservata scende leggermente, e sembrerebbe che la rete diventi meno omofila; in realtà scende ancora di più l'omofilia attesa, perché cambiano le proporzioni tra gli stati, e l'assortatività cresce fin dall'inizio. Una cautela analoga vale per la belief polarisation, che con questa codifica dipende solo dalle proporzioni di infetti e resistenti e non dalla struttura della rete.

In sintesi, la co-evoluzione aumenta la segregazione soprattutto sul piano delle opinioni, e usare solo le metriche strutturali avrebbe sottostimato il fenomeno. Va però detto che il rewiring considera solo coppie di nodi a distanza due, che appartengono quasi sempre alla stessa comunità: parte della crescita di ECI e modularità potrebbe dipendere da questo. Inoltre, con questi dati non è possibile separare l'effetto della selezione (rewiring) da quello dell'influenza (contagio).

5.4 Effetto dell'intervento

Tabella 3: Confronto allo step 129 tra la simulazione con 20 fact-checker e quella di controllo.

Grandezza	Intervento	Controllo	Differenza
Suscettibili	798	974	−176
Infetti	2 020	2 024	−4
Resistenti	2 162	2 002	+160
Fact-checker	20	0	+20
Transizioni negli step 100–129	177	17	+160
Belief assortativity	0.704	0.692	+0.012
Echo Chamber Index	0.886	0.886	0.000

Dalla Tabella 3 si vede che l'intervento agisce soprattutto sui suscettibili: escludendo i 20 nodi diventati fact-checker, 156 suscettibili in più rispetto al controllo diventano resistenti, circa il 16% di quelli rimasti. La differenza è circa nove volte il numero di transizioni spontanee del controllo, quindi non è dovuta al caso. La selezione aveva stimato una copertura di 162 nodi, molto vicina ai 160 resistenti in più osservati.

Sugli infetti, invece, l'effetto è trascurabile: 4 in meno, meno della variazione spontanea del controllo nello stesso periodo (8 nodi). Questo dipende dalle regole del modello, perché per recuperare un infetto serve che un quarto dei suoi vicini sia fact-checker, condizione che 20 nodi non riescono a raggiungere.

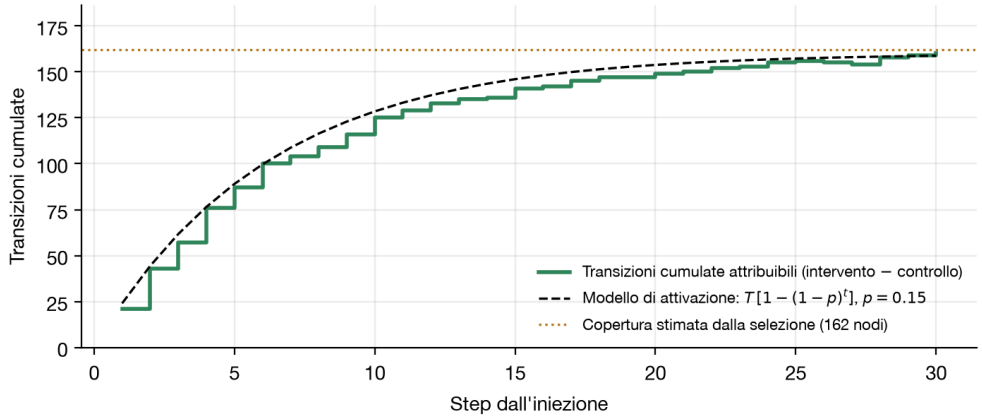


Figura 4: Transizioni cumulate attribuibili all'intervento (differenza tra le due simulazioni) e previsione di un modello che tiene conto solo della probabilità di attivazione dei nodi.

La Figura 4 mostra quando avviene l'effetto. Un nodo protetto può cambiare stato solo quando viene attivato, e ogni nodo è attivo con probabilità 0.15 a ogni step: dopo 10 step ci si aspetta quindi che sia stato attivato circa l'80% dei nodi coperti. Il valore osservato è il 78%, quindi la velocità dell'intervento dipende essenzialmente da quanto spesso gli agenti sono attivi.

ECI e modularità restano uguali nelle due simulazioni: l'intervento cambia le opinioni ma non la struttura della rete. In termini epidemiologici, il fact-checking in questa forma funziona come una vaccinazione mirata dei suscettibili più che come una cura degli infetti.

5.5 Tempi di esecuzione

Uno step richiede circa 6 minuti con due GPU T4 (circa 12 con una sola), e quasi tutto il tempo è occupato dalle chiamate all'LLM. La Fase 2 ha richiesto circa 10 ore, ciascuna simulazione

della Fase 3 circa 3 ore. Il tempo per scegliere i fact-checker è trascurabile.

6 Limiti e sviluppi futuri

Il progetto ha alcuni limiti di cui tenere conto:

- ogni configurazione è stata eseguita una sola volta, a causa del tempo richiesto, quindi non sono disponibili intervalli di confidenza: gli effetti grandi, come la protezione dei suscettibili, sono affidabili, quelli piccoli andrebbero confermati con più esecuzioni;
- vLLM non garantisce risposte perfettamente identiche quando cambia la composizione dei batch, quindi la riproducibilità non è completa;
- le metriche strutturali si riferiscono sempre alla partizione iniziale, e i profili degli agenti sono assegnati in base alla comunità in modo semplificato;
- una rete di co-autoraggio è solo un'approssimazione di una rete sociale, il campione ha perso gli hub più grandi e i risultati dipendono dal modello linguistico e dai prompt usati.

Come sviluppi futuri sarebbe utile ripetere le simulazioni con seed diversi, provare varianti del rewiring (disattivato o indipendente dalle opinioni) per separare selezione e influenza, e anticipare l'intervento o aumentarne il budget, per capire in quali condizioni il fact-checking riesce a ridurre anche il numero di infetti.

7 Conclusioni

Il progetto ha realizzato una simulazione in cui agenti LLM, struttura della rete e intervento evolvono insieme in un unico ciclo. Le scelte principali sono state un modello a soglia modulato dalle risposte dell'LLM, GraphSAGE per collegare le opinioni alla struttura della rete, un rewiring a densità costante e una selezione greedy dei fact-checker allineata alle regole della simulazione, preferita a CELF perché la funzione da ottimizzare non è submodulare.

Dai risultati emerge che la diffusione si ferma a circa il 40% della rete, con un andamento simile al modello SIR; che la co-evoluzione produce omofilia di opinione, assente all'inizio, molto più di quanto aumenti la segregazione tra comunità; e che un intervento tardivo con lo 0.4% dei nodi protegge una parte dei suscettibili, in misura prevedibile già prima di eseguirlo, ma non riduce il numero di infetti.

Riferimenti bibliografici

[1] N. Gabriele, M. Capalbo. *Appunti di Social Media and Network Analysis*. Materiale didattico, Università della Calabria, A.A. 2023/2024.

[2] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, E. Lefebvre. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, P10008, 2008.

[3] D. Centola, M. Macy. Complex contagions and the weakness of long ties. *American Journal of Sociology*, 113(3):702–734, 2007.

[4] M. Cinelli, G. De Francisci Morales, A. Galeazzi, W. Quattrociocchi, M. Starnini. The echo chamber effect on social media. *Proceedings of the National Academy of Sciences*, 118(9):e2023301118, 2021.

[5] M. Granovetter. Threshold models of collective behavior. *American Journal of Sociology*, 83(6):1420–1443, 1978.

[6] A. Grattafiori et al. The Llama 3 herd of models. arXiv:2407.21783, 2024.

[7] A. Grover, J. Leskovec. node2vec: Scalable feature learning for networks. In *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2016.

[8] W. L. Hamilton, R. Ying, J. Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[9] P. Holme, M. E. J. Newman. Nonequilibrium phase transition in the coevolution of networks and opinions. *Physical Review E*, 74:056108, 2006.

[10] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, J. Leskovec. Open Graph Benchmark: Datasets for machine learning on graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[11] D. Kempe, J. Kleinberg, É. Tardos. Maximizing the spread of influence through a social network. In *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 137–146, 2003.

[12] W. O. Kermack, A. G. McKendrick. A contribution to the mathematical theory of epidemics. *Proceedings of the Royal Society A*, 115(772):700–721, 1927.

[13] T. N. Kipf, M. Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

[14] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

[15] J. Leskovec, C. Faloutsos. Sampling from large graphs. In *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 631–636, 2006.

[16] J. Leskovec, A. Krause, C. Guestrin, C. Faloutsos, J. VanBriesen, N. Glance. Cost-effective outbreak detection in networks. In *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 420–429, 2007.

[17] D. Liben-Nowell, J. Kleinberg. The link-prediction problem for social networks. *Journal of the American Society for Information Science and Technology*, 58(7):1019–1031, 2007.

[18] M. McPherson, L. Smith-Lovin, J. M. Cook. Birds of a feather: Homophily in social networks. *Annual Review of Sociology*, 27:415–444, 2001.

[19] M. E. J. Newman. Mixing patterns in networks. *Physical Review E*, 67:026126, 2003.

[20] M. E. J. Newman. Modularity and community structure in networks. *Proceedings of the National Academy of Sciences*, 103(23):8577–8582, 2006.

[21] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[22] U. N. Raghavan, R. Albert, S. Kumara. Near linear time algorithm to detect community structures in large-scale networks. *Physical Review E*, 76:036106, 2007.

[23] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.